| | |
|---|---|
| **Challenge:** | Dawlish Leak |
| **Category:** | Multi-stage OSINT + Git Forensics + Cryptography problem |
| **Difficulty:** | Moderate |
| **Tool Used:** | Github, Git, CyberChef or Python (Used CyberChef here) |

# Summary

The Dawlish Leak challenge tests a mix of OSINT, Git forensics, and basic cryptography. We start by finding the GitHub profile **DawlishSec99** and sorting through multiple repositories, ignoring the decoys to identify the real one - Vault-Backup-Scripts.
By digging into the commit history and hidden branches, we uncover a deleted key ("LUMOS") and an encrypted payload. Using XOR decryption, we recover the final flag. Overall, the challenge shows how secrets can still leak through Git history and how weak encryption can be easily broken when combined with the right approach.

# Problem Statement

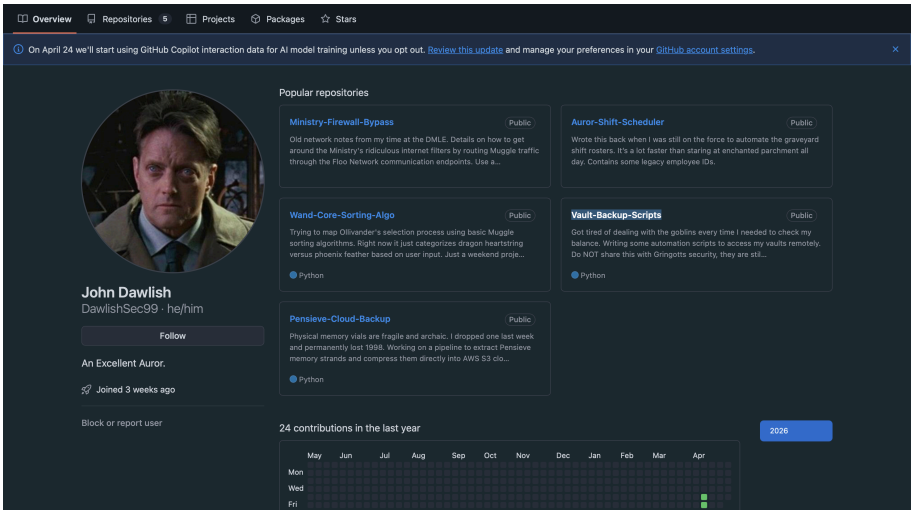
John Dawlish, a recently terminated Auror, has resurfaced in the digital underground. His activities suggest attempts to integrate magical systems into modern infrastructure, leaving behind a scattered and misleading digital trail.
Operating under the alias **DawlishSec99**, he maintains multiple public repositories—some genuine, others deliberately designed as decoys to mislead investigators.
Despite his efforts to cover his tracks, a critical mistake has been made. Sensitive information has been unintentionally exposed somewhere within his repositories.
The objective of this challenge is to analyse his digital footprint, identify the correct source of information, and uncover the hidden secret.
**Flag format:** CTF[your_flag_here]

# Steps to solve

**1)** OSINT & Repository Identification :
The solver begins by searching for the GitHub user **DawlishSec99**.
Upon exploring the profile, multiple repositories are found. However, not all of them are relevant - several are intentionally designed as decoys containing fake flags and misleading data.

By carefully analysing repository descriptions and ignoring distractions, the solver identifies the correct repository: **Vault-Backup-Scripts**

Hint Used : Hint 1 (Skip the decoys. Read the repository descriptions carefully and focus *only* on the one handling secure storage/backups.)



**2**) Git History Forensics : Clone the repository locally.

```
arpitpujani@Arpits-MacBook-Air-4 ~ % git clone https://github.com/DawlishSec99/Vault-Backup-Scripts.git
cd Vault-Backup-Scripts
Cloning into 'Vault-Backup-Scripts'...
remote: Enumerating objects: 43, done.
remote: Counting objects: 100% (43/43), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 43 (delta 6), reused 38 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (43/43), 4.92 KiB | 2.46 MiB/s, done.
Resolving deltas: 100% (6/6), done.
arpitpujani@Arpits-MacBook-Air-4 Vault-Backup-Scripts %
```

After cloning the repository locally, the solver inspects the commit history using: **git log -p**

```
commit c6494cdf074e254fedb793d5a54ec4ae985aa054 (HEAD -> main, origin/main, origin/HEAD)
Author: John Dawlish <pangameryom@gmail.com>
Date:   Fri Apr 10 07:27:20 2026 +0000

    Populated full vault directory structure

diff --git a/01_Authentication/biometrics/.gitkeep b/01_Authentication/biometrics/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/01_Authentication/oauth_flows/.gitkeep b/01_Authentication/oauth_flows/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/02_Vault_Mapping/deep_vaults_700s/.gitkeep b/02_Vault_Mapping/deep_vaults_700s/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/02_Vault_Mapping/level_1_retail/.gitkeep b/02_Vault_Mapping/level_1_retail/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/02_Vault_Mapping/level_2_commercial/.gitkeep b/02_Vault_Mapping/level_2_commercial/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/03_Cron_Jobs/daily_sync/.gitkeep b/03_Cron_Jobs/daily_sync/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/03_Cron_Jobs/emergency_lockdown/.gitkeep b/03_Cron_Jobs/emergency_lockdown/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/03_Cron_Jobs/weekly_audit/.gitkeep b/03_Cron_Jobs/weekly_audit/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/04_Logs/access_logs/.gitkeep b/04_Logs/access_logs/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/04_Logs/error_dumps/.gitkeep b/04_Logs/error_dumps/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/04_Logs/goblin_audit_trails/.gitkeep b/04_Logs/goblin_audit_trails/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/05_Dependencies/boto3_custom_fork/.gitkeep b/05_Dependencies/boto3_custom_fork/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/05_Dependencies/goblin_rest_api/.gitkeep b/05_Dependencies/goblin_rest_api/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/05_Dependencies/pycryptodome_magical/.gitkeep b/05_Dependencies/pycryptodome_magical/.gitkeep
new file mode 100644
index 0000000..e69de29
diff --git a/06_Utils/data_parsers/.gitkeep b/06_Utils/data_parsers/.gitkeep
new file mode 100644
index 0000000..e69de29
:
```

Although the current codebase appears clean, the commit history reveals previously deleted information. Within the 07_Legacy_Scripts directory, a deleted commit exposes a hardcoded encryption key: **XOR_KEY = “LUMOS"**
Hint Used : Hint 2 (The current code is clean, but Git remembers everything. Check the main branch commit history (git log -p) for a deleted encryption key.)

```
diff --git a/07_Legacy_Scripts/v2_unstable/vault_sync.py b/07_Legacy_Scripts/v2_unstable/vault_sync.py
index 71eb5ec..a0606dd 100644
--- a/07_Legacy_Scripts/v2_unstable/vault_sync.py
+++ b/07_Legacy_Scripts/v2_unstable/vault_sync.py
@@ -1,10 +1,7 @@
 import os

-# LEGACY ENCRYPTION KEY - TODO: REMOVE THIS BEFORE PUSHING
-# I used this to encrypt the old tokens via bitwise XOR.
-XOR_KEY = "LUMOS"
-
```

continue the investigation, the solver enumerates all branches using: **git branch -a**
This reveals a hidden branch: **hotfix/legacy-api**

```
arpitpujani@Arpits-MacBook-Air-4 Vault-Backup-Scripts % git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/hotfix/legacy-api
  remotes/origin/main
arpitpujani@Arpits-MacBook-Air-4 Vault-Backup-Scripts %
```

Switching to this branch and reviewing its history leads to the discovery of a deleted backup file inside the **10_Config_Templates** folder. This file contains an encrypted hex payload.

**0f010b34172d22212620240a217c67276629102724661239673939391038 7f2c30**

Hint Used :- Hint 3 (The encrypted payload isn't in main. Investigate hidden remote branches ( git branch -a) and check the deleted files in the hotfix branch history.)

**3**) Cryptographic Reversal :

The solver now possesses the Lock (the Hex String) and the Key (LUMOS), and knows the mechanism (XOR) based on Dawlish's deleted comments.

They utilise a tool like CyberChef (Hex to XOR) or write a custom Python script to perform a bitwise XOR operation against the payload.

The decryption successfully reveals the true flag: **CTF{Dawlish_l34k3d_th3_v4ult_k3y}**

Hint Used :- Hint 4 (You have the Hex payload and the Key (LUMOS). Dawlish used a simple bitwise XOR. Put them together in CyberChef to break the lock.)